# Documentation Technique

## SAE S6 - Projet Bibliothèque

| | |
|---|---|
| **Livrable** | L3 - Documentation technique |
| **Projet** | Application de gestion de bibliothèque |
| **Technologies** | Symfony 7.4 / Angular 21 / MariaDB 11 |
| **Année** | 2025-2026 - Semestre 6 |
| **Formation** | BUT Informatique - IUT de Blagnac |
| **Groupe** | 3B - Yassine Badaoui - Ugo Pia - Melvin Bouyssou |

# Table des matières


| | | |
|---|---|---|
| **1.** | **Architecture du projet** | **3** |
| 1.1 | Schéma global de l'architecture | 3 |
| 1.2 | Diagramme de classes | 4 |
| 1.3 | Architecture front-end Angular | 5 |
| 1.4 | Sécurité et authentification | 5 |
| **2.** | **Choix techniques** | **6** |
| 2.1 | Stack technologique | 6 |
| 2.2 | Structure des entités et ORM | 6 |
| 2.3 | Gestion des réservations | 7 |
| 2.4 | Gestion des emprunts | 7 |
| 2.5 | Recherche et filtrage | 8 |
| 2.6 | Back-office EasyAdmin | 8 |
| **3.** | **Guide d'installation** | **9** |
| 3.1 | Prérequis | 9 |
| 3.2 | Installation pas à pas | 9 |
| 3.3 | Identifiants de test | 10 |
| **4.** | **Difficultés rencontrées et solutions** | **11** |
| **5.** | **Répartition du travail** | **12** |

# 1. Architecture du projet

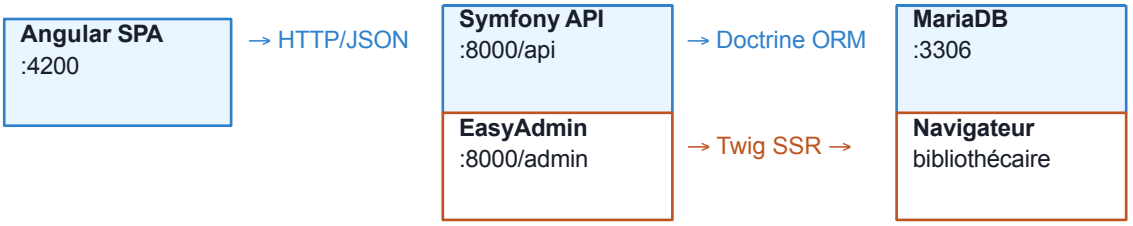
## 1. Schéma global de l'architecture

L'application SAE Biblio suit une architecture **trois tiers** classique, séparant clairement la couche de présentation (front-end), la logique métier (API back-end) et la persistance des données (base de données). Un back-office d'administration vient compléter le dispositif.

| Couche | Technologie | Rôle |
|---|---|---|
| Front-end (Client SPA) | Angular 21 TypeScript / Signals | Interface utilisateur, navigation, authentification JWT, appels API REST |
| API REST (Back-end) | Symfony 7.4 PHP 8.2+ | Logique métier, validation, sérialisation JSON, sécurité |
| Back-office (Administration) | EasyAdmin 5 + Twig | CRUD catalogue, gestion membres, suivi emprunts/réservations |
| Base de données (Persistance) | MariaDB 11 (Docker) | Stockage relationnel, intégrité référentielle, index performants |

**Flux de communication :** Le front Angular communique exclusivement avec le back-end Symfony via des requêtes HTTP REST (JSON). Le back-office EasyAdmin est servi directement par Symfony en Server-Side Rendering (Twig). L'authentification API utilise des tokens JWT (LexikJWTBundle).

**Schéma des flux :**

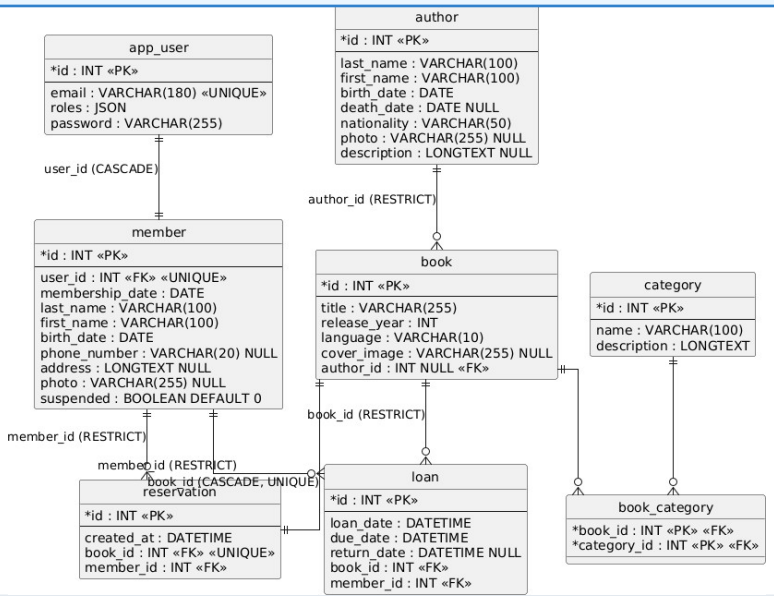


## 2. Diagramme de classes (Entités Doctrine)

Le modèle de données repose sur **7 entités Doctrine** interconnectées. Voici le détail de chaque entité avec ses attributs et relations :

| Entité | Attributs | Relations |
|---|---|---|
| **User** | id (int PK), email (varchar 180, unique), roles (json), password (varchar 255) | OneToOne → Member |
| **Member** | id (int PK), user_id (FK unique), lastName (100), firstName (100), birthDate (date), membershipDate (date), phoneNumber (20), address (text), photo (255), suspended (bool) | OneToOne ← User OneToMany ← Loan OneToMany ← Reservation |
| **Book** | id (int PK), title (255), releaseYear (int), language (10), coverImage (255), author_id (FK nullable) | ManyToOne → Author<br>ManyToMany → Category<br>OneToOne ← Reservation<br>OneToMany ← Loan |
| **Author** | id (int PK), lastName (100), firstName (100), birthDate (date), deathDate (date nullable), nationality (50), photo (255), description (text) | OneToMany ← Book |
| **Category** | id (int PK), name (100), description (text) | ManyToMany ← Book (via book_category) |
| **Loan** | id (int PK), book_id (FK), member_id (FK), loanDate (datetime), dueDate (datetime), returnDate (datetime nullable) | ManyToOne → Book<br>ManyToOne → Member |
| **Reservation** | id (int PK), book_id (FK unique), member_id (FK), createdAt (datetime) | OneToOne → Book<br>ManyToOne → Member |

**Contraintes d'intégrité notables :** La table `reservation` impose une contrainte UNIQUE sur `book_id` (un livre ne peut être réservé qu'une fois). Les suppressions de livres sont bloquées par `ON DELETE RESTRICT` sur les emprunts, évitant toute perte de traçabilité. La table pivot `book_category` utilise `ON DELETE CASCADE` pour le nettoyage automatique.



## 3. Architecture front-end Angular

Le front-end est une **Single Page Application (SPA)** Angular 21 utilisant les dernières fonctionnalités du framework : standalone components, signals pour la réactivité, et lazy loading des routes.

| Dossier / Fichier | Rôle |
|---|---|
| components/public/ | Pages accessibles sans authentification : accueil, catalogue, détail livre, auteurs |
| components/auth/ | Page de connexion (login) |
| components/member/ | Espace adhérent : tableau de bord, profil |
| components/layout/ | Composants transversaux : navbar, footer |
| services/ | AuthService (JWT), BookService (API), MemberService (profil/prêts) |
| guards/ | authGuard (route protégée), roleGuard (contrôle de rôle) |
| interceptors/ | jwtInterceptor : injection automatique du token Bearer |
| models/ | Interfaces TypeScript (Book, Loan, Member, Reservation, etc.) |

## 4. Sécurité et authentification

L'application implémente un système de sécurité multi-couches basé sur les rôles Symfony :

| Rôle | Hérite de | Accès |
|---|---|---|
| ROLE_USER | - | Authentification de base |
| ROLE_MEMBRE | ROLE_USER | Espace adhérent : réservations, emprunts, profil |
| ROLE_BIBLIOTHECAIRE | ROLE_MEMBRE | Enregistrement emprunts/retours, recherche membres |
| ROLE_RESPONSABLE | ROLE_BIBLIOTHECAIRE | Back-office complet, statistiques, gestion catalogue |

L'API utilise **LexikJWTAuthenticationBundle** pour l'authentification stateless par token JWT. Le back-office EasyAdmin utilise un authenticator custom avec sessions et remember-me. Un interceptor Angular injecte automatiquement le header `Authorization: Bearer` sur chaque requête API.

# 2. Choix techniques

## 1. Stack technologique

| Composant | Technologie | Version | Justification |
|---|---|---|---|
| Framework back | Symfony | 7.4 | Framework PHP mature, écosystème riche (Doctrine, Security, Serializer) |
| ORM | Doctrine ORM | 3.6 | Mapping objet-relationnel robuste, migrations automatiques, DQL puissant |
| API REST | Contrôleurs Symfony | - | Endpoints REST manuels (pas de génération automatique), contrôle fin des routes et réponses JSON |
| Auth API | LexikJWT | 3.2+ | Standard JWT, stateless, compatible Angular |
| Back-office | EasyAdmin | 5.0 | Interface CRUD rapide, personnalisable, intégrée Symfony |
| Front-end | Angular | 21 | Framework structuré, TypeScript natif, signals pour la réactivité |
| BDD | MariaDB | 11 | Compatible MySQL, performant, conteneurisé via Docker |
| CORS | NelmioCorsBundle | 2.6+ | Gestion fine des origines cross-domain pour le SPA |
| Conteneurisation | Docker Compose | - | Isolation de la BDD, reproductibilité de l'environnement |

## 2. Structure des entités et ORM

Les entités Doctrine utilisent les **attributs PHP 8** (annotations modernes) pour le mapping ORM. Chaque entité utilise les groupes de sérialisation Symfony (`#[Groups]`) pour contrôler finement les champs exposés par l'API selon le contexte : `book:list` (catalogue), `book:read` (détail), `loan:read` (emprunts), `reservation:read` (réservations).

La validation est assurée par les contraintes Symfony (`#[Assert]`) directement sur les propriétés : `NotBlank`, `Length`, `Range`, `Email`, `UniqueEntity`. Cela garantit l'intégrité des données avant même la persistance en base.

## 3. Gestion des réservations

Le système de réservation implémente les règles métier suivantes :

- **Limite à 3 réservations simultanées** par adhérent - vérifiée côté API via `countByMember()`.
- **Unicité par livre** - contrainte UNIQUE sur `book_id` dans la table `reservation`, empêchant toute double réservation.
- **Interdiction de réserver un livre emprunté** - vérification de l'absence d'emprunt actif via `findActiveByBookId()`.
- **Expiration automatique à J+7** - implémentée dans le `ReservationRepository` : chaque requête de lecture appelle `deleteExpired()` qui supprime les réservations dont `createdAt < now - 7 jours`. Ce mécanisme "lazy" évite le recours à un cron job.
- **Annulation libre** - un adhérent peut annuler ses propres réservations via `DELETE /api/me/reservations/{id}`.
- **Compte suspendu** - les membres suspendus (`suspended = true`) reçoivent une erreur 403 lors de toute tentative de réservation.

**Choix d'implémentation : expiration lazy.** Plutôt que d'utiliser un cron ou une commande Symfony planifiée, l'expiration des réservations est gérée "à la volée" dans le Repository. Chaque appel à `findByMember()` ou `findOneByBookId()` filtre automatiquement les réservations expirées et les purge. Cela simplifie le déploiement et garantit une cohérence immédiate des données à chaque lecture.

## 4. Gestion des emprunts

La logique métier des emprunts est centralisée dans un **service dédié** (`LoanService`) qui encapsule toutes les règles :

- **Durée maximale 15 jours** - calculée automatiquement dans le constructeur de `Loan` (`dueDate = loanDate + 15 jours`).
- **Quota d'emprunts** - paramètre configurable via `LibraryRules.php` qui contient les constantes des contraintes métier.
- **Vérification de disponibilité** - `isBookAvailable()` vérifie l'absence d'emprunt actif (returnDate IS NULL).
- **Conversion réservation → emprunt** - lors de `registerLoan()`, si une réservation existe pour le livre, elle est automatiquement supprimée.
- **Gestion des conflits** - si le livre est réservé par un autre adhérent, l'API retourne un warning avec `requireConfirmation: true`, permettant au bibliothécaire de forcer l'emprunt (`force: true`).
- **Détection des retards** - les requêtes `findOverdueLoans()` comparent `dueDate` avec la date courante pour identifier les emprunts en retard.

## 5. Recherche et filtrage

La recherche de livres est implémentée via un **QueryBuilder dynamique** dans `BookRepository::createSearchQueryBuilder()`. Les filtres sont composables et combinables : titre (LIKE), auteur (ID), catégories (IN, jusqu'à 3), langue, et plage d'années.

La pagination utilise une stratégie en deux requêtes pour éviter les problèmes de `DISTINCT` avec les jointures ManyToMany : une première requête récupère les IDs paginés, puis une seconde charge les entités complètes avec leurs relations (auteur, catégories) via `findByIdsWithAuthorAndCategories()`.

Côté Angular, le composant `CatalogueComponent` utilise les **signals** pour la réactivité et un service `SearchFiltersStorageService` qui persiste les filtres de recherche dans le `sessionStorage`, permettant de conserver les critères lors de la navigation.

## 6. Back-office EasyAdmin

Le back-office utilise **EasyAdmin 5** avec des CRUD Controllers dédiés pour chaque entité : AuthorCrud, BookCrud, CategoryCrud, LoanCrud, MemberCrud, ReservationCrud, UserCrud. Le DashboardController expose des statistiques agrégées (nombre de livres, membres, emprunts, réservations). L'accès est restreint via `access_control` : seuls les utilisateurs avec `ROLE_BIBLIOTHECAIRE` ou supérieur peuvent accéder à `/admin`. Le menu est conditionnel : les sections de gestion catalogue et membres ne sont visibles qu'au `ROLE_RESPONSABLE`.

# 3. Guide d'installation

## 1. Prérequis

| Outil | Version minimale | Vérification |
|---|---|---|
| Git | 2.x | git --version |
| PHP | 8.2 | php -v |
| Composer | 2.x | composer --version |
| Node.js | 20.x | node -v |
| npm | 10.x | npm -v |
| Docker + Compose | 24.x / 2.x | docker compose version |
| Symfony CLI | 5.x (recommandé) | symfony version |

## 2. Installation pas à pas

### Étape 1 - Cloner le dépôt et configurer l'environnement

```
git clone https://github.com/ez1kl/sae-s6.git
cd sae-s6
cp .env.example .env
```

### Étape 2 - Lancer la base de données MariaDB

```
docker compose up -d
# Vérifier le statut :
docker compose ps
```

### Étape 3 - Installer et configurer le back-end Symfony

```
cd biblio
cp .env.exemple .env
composer install
php bin/console doctrine:database:create --if-not-exists
php bin/console doctrine:migrations:migrate -n
php bin/console doctrine:fixtures:load -n
php bin/console lexik:jwt:generate-keypair # si pas déjà fait
```

### Étape 4 - Installer et lancer le front-end Angular

```
cd ../front-biblio
npm install
# Adapter src/app/environments/environment.ts si besoin
npm start
```

### Étape 5 - Lancer le serveur Symfony

```
cd ../biblio
symfony server:start # ou php -S 127.0.0.1:8000 -t public/
```

**Points d'accès :**
- Front Angular : `http://localhost:4200`
- API Symfony : `http://localhost:8000/api`
- Back-office : `http://localhost:8000/admin`

## 3. Identifiants de test (fixtures)

| Profil | Email | Mot de passe | Rôle |
|---|---|---|---|
| Responsable | admin@admin.fr | admin | ROLE_RESPONSABLE |
| Bibliothécaire | biblio@biblio.fr | biblio | ROLE_BIBLIOTHECAIRE |
| Adhérent (x10) | Généré par Faker | user | ROLE_MEMBRE |

Les emails des adhérents sont générés aléatoirement par Faker lors du chargement des fixtures. Pour les récupérer, consulter la table `app_user` en base de données. Les fixtures créent également 5 auteurs, 5 catégories, 20 livres, 15 emprunts variés (rendus, en cours, en retard) et 3 réservations.

# 4. Difficultés rencontrées et solutions apportées

| Difficulté | Contexte | Solution apportée |
|---|---|---|
| Pagination avec jointures ManyToMany | La pagination classique avec LIMIT/OFFSET produit des résultats incorrects quand un livre a plusieurs catégories (doublons dans le COUNT). | Stratégie en deux requêtes : d'abord récupérer les IDs distincts paginés, puis charger les entités complètes avec les jointures. Cela évite les problèmes de DISTINCT avec LIMIT. |
| Expiration des réservations sans cron | La règle métier impose l'expiration à J+7, mais l'ajout d'un cron job complexifie le déploiement. | Implémentation d'un mécanisme "lazy" : chaque lecture dans le ReservationRepository filtre et purge automatiquement les réservations expirées, rendant le système autonome. |
| Conflit réservation /emprunt | Gérer le cas où un bibliothécaire veut prêter un livre réservé par un autre adhérent sans bloquer complètement l'opération. | Système de warning avec confirmation : l'API retourne un code 409 avec requireConfirmation=true. Le front affiche un message d'avertissement et permet au bibliothécaire de forcer l'emprunt (paramètre force). |
| Authentification JWT + EasyAdmin | Cohabitation de deux systèmes d'auth : JWT stateless pour l'API et sessions pour le back-office EasyAdmin. | Configuration de deux firewalls Symfony distincts : 'api' (JWT stateless) et 'main' (sessions + AdminAuthenticator custom). Le firewall 'login' est isolé pour l'endpoint /api/login. |

| Difficulté | Contexte | Solution apportée |
|---|---|---|
| Sérialisation et références circulaires | Les relations bidirectionnelles Doctrine provoquent des boucles infinies lors de la sérialisation JSON. | Utilisation des groupes de sérialisation Symfony (#[Groups]) pour contrôler précisément les champs exposés par contexte, évitant les références circulaires tout en gardant les relations utiles. |

# 5. Répartition du travail dans le groupe

Le tableau ci-dessous détaille la répartition des responsabilités entre les trois membres du groupe.

| Membre | Responsabilités principales | Livrables associés |
|---|---|---|
| Yassine Badaoui | - Front-end Angular (composants, services)<br>- UX/UI catalogue et espace adhérent<br>- Configuration sécurité (JWT, firewalls)<br>- Back Office interface libraire<br>- Documentation technique et API | Angular (catalogue, dashboard, profil, auteurs), API.md, README.md, ce documentComposants |
| Ugo Pia | - API REST (Controllers API)<br>- Service LoanService et logique métier<br>- Migrations, fixtures et données de test<br>- Recherche et filtrage front-end | BookController, MemberController, LibrarianController, AdminStatsController, LoanService, security.yaml, AppFixtures, modèles TypeScript |
| Melvin Bouyssou | - Modélisation BDD et entités Doctrine<br>- Authentification JWT<br>- Back-office EasyAdmin (CRUD)<br>- Configuration Docker (MariaDB) | Entités PHP, migrations, docker-compose.yml, CRUD Controllers EasyAdmin, DashboardController , services, guards, interceptors, |